

DLBDSOOFPP01

Object Oriented and functional programming with Python

Task : Create a Habit Tracking App

Development Phase

Arrehquette Ewube Eyong

IU International Hochschule

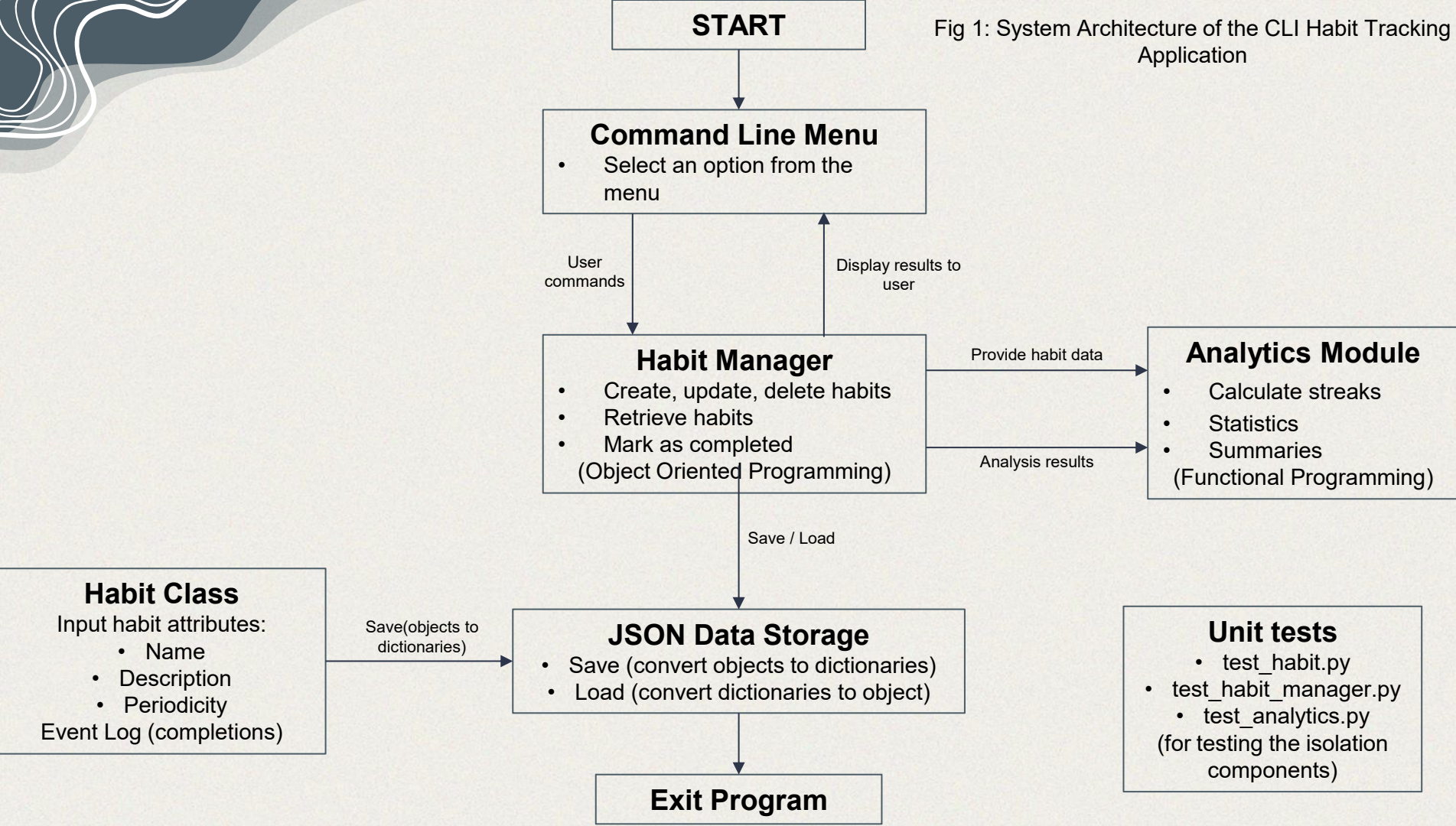
Table of contents

- Project Overview
- System Architecture
- Habit Class
- Habit Manager
 - Predefined Habits & Four-Week Test Data
- Analytics Functions
- Persistence Layer
- Testing and Results
- Command Line Interface
- Conclusion



Project Overview

- The Habit Tracker is a command line application built in Python that helps users build and maintain daily and weekly habits. The system enables users to create, edit, delete and complete habits while recording their progress over time
- The application is designed with a modular architecture, with each module responsible for a specific task. The Habit class models individual habits, the Habit Manager manages habit-related operations, the Analytics module performs statistical analysis and streak calculations. The user data is stored persistently in a JSON file while Unit test files verify the functionality of the components of the app and the main.py file serves as the application's entry point and command line interface
- The project was implemented using the following key technologies:
 - Python – programming language
 - Object-Oriented Programming – organize habits and core components in a structured way
 - Functional Programming – carries out habit analysis and streak calculations
 - JSON – the persistent storage for habit data
 - Pytest – for automated unit testing



Habit Class

- The Habit Class represents a single habit the user wants to track. Each habit is stored as an object containing its own information, current streak, longest streak and completion history. This object-oriented approach allows each habit maintain its own state and behavior independently.

Each habit includes a name, description, periodicity, creation date and completion history, providing the information needed to manage the habits and evaluate user progress.

Every time a habit is completed, a timestamp is recorded to its event log. This structure preserves a full history of activity and forms the basis for streak and progress calculation.

```
class Habit: 37 usages
    def __init__(
        self,
        name: str,
        description: str,
        periodicity: str,
        created_at: datetime = None,
        completions: List[str] = None,
    ):
```

Fig 2: Entry of habit details

- When all the important information about the habit is entered, the habit object is converted to dictionaries for JSON storage

Habit Manager – Object-Oriented programming

- The Habit Manager class handles the management of all habit operations. It creates, modifies, deletes, mark habits as completed and retrieve habits ensuring interaction between the habit object and the persistence storage
- It manages data persistence by saving and loading habit information from the persistence storage, ensuring the user's progress is maintained even after restarting the application
- This class applies object-oriented programming principles like encapsulating data logic in one place and using composition to handle multiple habit objects
- Some methods used here are:
 - ✓ `add_habit()`
 - ✓ `update_habit()`
 - ✓ `delete_habit()`
 - ✓ `mark_complete()`
 - ✓ `find_habit()`
 - ✓ `load() / save()`

Predefined Habits & Four-Week Test Data

- The application includes predefined habits to support the development, testing and demonstration processes.
- These habits represent a realistic user activity over a four-week period, with sufficient sample data that allows the analytical functions to be tested and verified without requiring users to manually enter data.

```
Habit( name: "Take a nap",  
      description: "Nap for an hour",  
      periodicity: "daily",  
      completions=generate_dates( period_days: 1, count: 28)),  
  
Habit( name: "Exercise",  
      description: "Go to the gym",  
      periodicity: "weekly",  
      completions=generate_dates( period_days: 7, count: 4)),
```

Fig 3: Predefined habits in habit_manager.py

```
===Your habits===  
+-----+-----+  
| Habit Name | Periodicity |  
+=====+=====+  
| Take a nap | daily      |  
+-----+-----+  
| Exercise   | weekly     |  
+-----+-----+  
| Read a book | daily     |  
+-----+-----+  
| Call parents | weekly    |  
+-----+-----+  
| Clean room  | weekly     |  
+-----+-----+
```

Fig 4: Predefined habits as shown through the CLI

Analytics Functions (Functional Programming)

- The analytics module applies Functional Programming concepts including the use of pure functions, higher-order functions and declarative techniques to process and analyze data in a structured manner. The following functions were implemented to perform different habit analysis:
 - ✓ `map()` is used to perform calculations across habits. For example, determining the longest streak achieved
 - ✓ lambda expression enables the creation of anonymous functions for filtering and transforming habit data
 - ✓ `max()` with a key function is used to find the habit with the highest value according to the custom criterion. For example, finding the longest streak and most completed habit
 - ✓ Generator expressions work with functions like `sum()` and `max()` to calculate aggregated analytical results while minimizing memory usage
- These functional technique ensures the application efficiently process, filter and summarize habit data which keeps the code readable, reduce complexity and simplifies long term maintenance

Persistence Layer

- The Habit Tracker uses a JSON file as its persistent storage mechanism, as it is lightweight, human readable and stores structured data without requiring a database.
- As the user interacts with the application through the CLI, the requests are sent to the Habit Manager. The Habit Manager processes the operations (such as creating, updating, deleting or completing habits) before converting each habit each Habit object into a dictionary and saving in the JSON file. When the application starts, the stored dictionaries are loaded from the JSON file and converted back into Habit object.

```
"name": "Take a nap",  
"description": "Nap for an hour",  
"periodicity": "daily",  
"created_at": "2026-07-27T23:47:54.120620",  
"completions": [  
    "2026-07-28T00:04:44.350428"  
]  
  
"name": "Exercise",  
"description": "Go to the gym",  
"periodicity": "weekly",  
"created_at": "2026-07-27T23:47:54.120620",  
"completions": []
```

Fig 5: JSON Storage

Testing and Results

Automated unit tests were developed using pytest framework to ensure the Habit Tracker produce correct result and if it is reliable. The testing was divided into two dedicated modules:

a. test_habit.py for testing the core functionality of the Habit object. It tested the following areas:

- Creation of valid habits
- Validation of empty habit names
- Validation of invalid periodicity
- Recording habit completions
- Calculation of current streaks
- Calculation of longest streaks
- Resetting the current streak after a missed completion day

```
===== test session starts =====
collecting ... collected 7 items

test_habit.py::test_create_valid_habit PASSED [ 14%]
test_habit.py::test_empty_habit PASSED [ 28%]
test_habit.py::test_invalid_periodicity PASSED [ 42%]
test_habit.py::test_mark_complete PASSED [ 57%]
test_habit.py::test_current_streak PASSED [ 71%]
test_habit.py::test_longest_streak PASSED [ 85%]
test_habit.py::test_current_streak_reset_after_missed_day PASSED [100%]

===== 7 passed in 0.01s =====
```

Fig 6: Test results for test_habit.py

Testing and Results

b. test_habit_manager.py tests the functionality of the Habit Manager which controls the creation, modification, deletion and storage of habits. The following operations were tested:

- Adding new habits
- Preventing duplicate habit creation
- Updating existing habit information
- Deleting habits
- Completing habits
- Saving and loading habit data from the JSON storage

```
===== test session starts =====
collecting ... collected 6 items

test_habit_manager.py::test_add_habit PASSED [ 16%]
test_habit_manager.py::test_add_duplicate_habit PASSED [ 33%]
test_habit_manager.py::test_update_habit PASSED [ 50%]
test_habit_manager.py::test_delete_habit PASSED [ 66%]
test_habit_manager.py::test_complete_habit PASSED [ 83%]
test_habit_manager.py::test_save_load PASSED [100%]

===== 6 passed in 0.07s =====
```

Fig 7: Test results for test_habit_manager.py

```
✓ 10 tests passed 10 tests total, 0 ms
test_analytics.py::test_total_habits PASSED
test_analytics.py::test_longest_streak PASSED
test_analytics.py::test_get_longest_streak_all PASSED
test_analytics.py::test_get_longest_streak_for_habit PASSED
test_analytics.py::test_get_current_streaks PASSED
test_analytics.py::test_get_habit_summary PASSED
test_analytics.py::test_get_habits_by_periodicity PASSED
test_analytics.py::test_count_by_periodicity PASSED
test_analytics.py::test_most_completed_habit PASSED
test_analytics.py::test_total_completions PASSED

===== 10 passed in 0.03s =====
```

Fig 8: Test results for test_analytics.py

c. test_analytics.py tests to verify that the habit data is processed and analyzed correctly. It ensures the implemented functional techniques produce accurate results

All tests passed successfully, confirming that the application's core features operate correctly. This automated testing improves reliability, identify errors early and ensure the application is stable

Command Line Interface (User Workflow)

- The Habit tracker is operated through a straightforward menu-driven command-line interface. The user interacts with the system by selecting menu options and entering the requested information necessary for each operation.
- The application enables users to:
 - ✓ Add a new habit by specifying the name, description and periodicity
 - ✓ Edit an existing habit's name, description or periodicity
 - ✓ Mark a habit as complete
 - ✓ View all habits in a table showing details and streak data
 - ✓ View habit analysis, including total habits, current/longest streak and daily/weekly counts
 - ✓ Delete unwanted habits
 - ✓ Exit the application safely, with all changes saved

Command Line Interface

- The use of colors and tables improves readability, making information easier to understand while maintaining a clean and organized interface.

```
=== Welcome to the Habit Tracker===  
1. Add habit  
2. Update habit  
3. Checklist  
4. View habits  
5. Analyze habits  
6. Delete habit  
7. Exit  
  
Enter your choice:
```

Fig 9: Main Menu (CLI)

- The CLI provides a simple, efficient and straightforward user experience. It separates the user interface from the application logic which improves modularity and maintainability, making it easier to replace the CLI with a graphical user interface in future versions of the application.

Conclusion

The objectives established during the conception phase have been successfully implemented throughout the development.

Compared to the initial conception phase, the development phase has been significantly enhanced through the inclusion of unit testing and more detailed specification of the individual modules.

These improvements increased the reliability, maintainability and documentation of the application.

Figures

- Figure 1: System Architecture of the CLI Habit Tracking Application
- Figure 2: Entry of habit details
- Figure 3: Predefined habits in habit_manager.py
- Figure 4: Predefined habits as shown through the CLI
- Figure 5: JSON Storage
- Figure 6: Test results for test_habit.py
- Figure 7: Test results for test_habit_manager.py
- Figure 8: Test results for test_analytics.py
- Figure 9: Main Menu (CLI)